

Express

Building a REST API with Node

CSCI 39548 — Practical Web Development

Section Goals

- ▶ Understand what a backend does and why you need one
- ▶ Build a REST API with Express and TypeScript
- ▶ Use middleware for logging, parsing, and error handling
- ▶ Validate request data with Zod
- ▶ Configure CORS for cross-origin requests

Section Items

- ▶ What Is a Backend?
- ▶ Setup
- ▶ REST CRUD
- ▶ Middleware and Structure
- ▶ Validation with Zod
- ▶ CORS and Serving the Frontend

► What Is a Backend?

- Setup
- REST CRUD
- Middleware and Structure
- Validation with Zod
- CORS and Serving the Frontend

What Is a Backend?

- ▶ A **server** that listens for HTTP requests, runs logic, and responds
- ▶ Holds **shared state** (the database) that all users see
- ▶ Enforces **rules** that you can't trust the browser to enforce (auth, validation, business logic)
- ▶ The frontend is one user's view; the backend is the source of truth

Why Node + Express?

	Node + Express	Alternatives
Language	JavaScript / TypeScript	Python, Go, Rust, Ruby
Win	Same language as frontend	Better perf (Go), batteries (Django)
Tradeoff	Single-threaded	Different language to switch into

Express is the minimal, ubiquitous Node web framework. Industry default for “we need a Node REST API.”

REST in One Slide

Resource = a noun

/api/todos, /api/users

HTTP verb = action on the noun:

- ▶ GET — read
- ▶ POST — create
- ▶ PUT — replace (full update)
- ▶ PATCH — partial update
- ▶ DELETE — delete

Status codes:

- ▶ 2xx success (200, 201, 204)
- ▶ 4xx client error (400, 401, 403, 404)
- ▶ 5xx server error (500, 502, 503)

Section Items

- What Is a Backend?

► **Setup**

- REST CRUD
- Middleware and Structure
- Validation with Zod
- CORS and Serving the Frontend

Hello Express

```
import express from "express";

const app = express();

app.get("/api/health", (req, res) => {
  res.json({ status: "ok" });
});

app.listen(3000);
```

The Pieces

- ▶ `app` is a function-routing object
- ▶ `app.get(path, handler)` registers a handler for GET requests
- ▶ `req, res` — the request and response objects
- ▶ `res.json(obj)` sets `Content-Type: application/json` and serializes

tsx watch

```
npx tsx watch src/server.ts
```

- ▶ Runs TypeScript directly — no build step
- ▶ Restarts on file change
- ▶ The productive workflow for development

Section Items

- What Is a Backend?
- Setup

► **REST CRUD**

- Middleware and Structure
- Validation with Zod
- CORS and Serving the Frontend

The Full Route Set

```
app.use(express.json());                // parse JSON bodies

app.get("/api/todos", listHandler);
app.get("/api/todos/:id", getOneHandler);
app.post("/api/todos", createHandler);
app.patch("/api/todos/:id", updateHandler);
app.delete("/api/todos/:id", deleteHandler);
```

Request Data Sources

- ▶ `req.params.id` — from the URL path (`/:id`)
- ▶ `req.query.filter` — from the query string (`?filter=done`)
- ▶ `req.body` — from the JSON body (only after `express.json()`)

Status Codes by Handler

Handler	Success	Failure
GET list	200	rarely
GET one	200	404
POST	201	400
PATCH	200	400, 404
DELETE	204	404

Live Code: REST CRUD

- ▶ Open the API in the browser: `http://localhost:3000/api/todos`
- ▶ Use the frontend to add, toggle, delete
- ▶ Watch the Network tab — every action is an HTTP request

Section Items

- What Is a Backend?
- Setup
- REST CRUD
- ▶ **Middleware and Structure**
 - Validation with Zod
 - CORS and Serving the Frontend

A Middleware Is a Function

```
function logger(req, res, next) {  
  console.log(`${req.method} ${req.url}`);  
  next();    // pass control on  
}  
  
app.use(logger);
```

`next()` passes control to the next middleware or route handler. Without it, the request hangs.

The Request Pipeline

```
request -> [logger] -> [json parser] -> [routes]  
        -> [404 handler] -> [error handler] -> response
```

- ▶ Order is registration order
- ▶ Logger before routes (so all routes log)
- ▶ 404 handler after routes (catches unmatched paths)
- ▶ Error handler last

The 4-Arg Error Handler

```
function errorHandler(err, req, res, next) {  
  console.error(err.stack);  
  res.status(500).json({ error: "Internal server error" });  
}  
  
app.use(errorHandler);
```

- ▶ Express uses **arity** (number of arguments) to detect error handlers
- ▶ Must have exactly 4 parameters — drop one and it's no longer an error handler

Routers — Files Don't Grow Forever

```
// routes/todos.ts
const router = Router();
router.get("/", listHandler);
router.post("/", createHandler);
export default router;
```

```
// server.ts
app.use("/api/todos", todoRouter);
```

Big app → many routers, each in its own file.

- ▶ Watch the logger print every request in the terminal
- ▶ Hit `/api/nonexistent` — 404 handler catches it
- ▶ Throw an error in a route — error handler catches it

Section Items

- What Is a Backend?
- Setup
- REST CRUD
- Middleware and Structure
- ▶ **Validation with Zod**
 - CORS and Serving the Frontend

Why Validate?

- ▶ The client lies. Bad data shouldn't crash the server.
- ▶ Hand-rolled `typeof` checks get long and miss edge cases (negative numbers, empty strings, etc.)
- ▶ A schema library declares the rules **once**, validates, and gives you a TypeScript type for free

A Zod Schema

```
import { z } from "zod";

export const TodoCreateSchema = z.object({
  text: z.string().trim().min(1).max(200),
  minutes: z.number().int().min(1).max(24 * 60),
});

export type TodoCreateInput = z.infer<typeof TodoCreateSchema>;
```

Declare the rules once. Get validation AND a TypeScript type.

Use in a Route

```
router.post("/", (req, res) => {
  const result = TodoCreateSchema.safeParse(req.body);
  if (!result.success) {
    res.status(400).json({
      error: "Validation failed",
      issues: result.error.flatten(),
    });
    return;
  }
  // result.data is typed AND validated
  const todo = todoStore.create(result.data.text, result.data.
    minutes);
  res.status(201).json(todo);
});
```

.parse vs .safeParse

Method	On invalid data	Use when
<code>.parse(data)</code>	Throws <code>ZodError</code>	Global error handler catches it
<code>.safeParse(data)</code>	Returns <code>{ success, data error }</code>	Handle inline

- ▶ **Zod** — current default in the TypeScript ecosystem
- ▶ **Joi** — older, JavaScript-only
- ▶ **Yup** — smaller, common in older React projects
- ▶ **Valibot** — newer, smaller bundles
- ▶ Same idea across all of them — Zod won the TS ecosystem

Live Code: Validation

- ▶ Submit an empty description — 400 with validation errors
- ▶ Submit a negative minutes value — 400
- ▶ Submit valid data — 201 Created

Section Items

- What Is a Backend?
- Setup
- REST CRUD
- Middleware and Structure
- Validation with Zod
- ▶ **CORS and Serving the Frontend**

Serving a Static Frontend

```
app.use(express.static("public"));
```

- ▶ `public/index.html` → served at `/`
- ▶ Express does double duty: API + static UI from one process

Why CORS Exists

- ▶ Browsers enforce **same-origin policy** — JS on `evil.com` can't fetch from `bank.com`
- ▶ CORS is the server's way to say "I trust this other origin to call me"

When You Need CORS

Frontend	Backend	CORS?
localhost:3000	localhost:3000	No — same origin
app.vercel.app	api.render.com	YES
Served by express.static	Same Express	No

Configuring CORS

```
import cors from "cors";

// Development (open to all)
app.use(cors());

// Production (lock it down)
app.use(cors({ origin: "https://app.vercel.app" }));
```

What CORS Does NOT Do

- ▶ Doesn't replace auth — anyone whose origin is allowed can still hit you
- ▶ Doesn't replace input validation — same client can still send junk
- ▶ Doesn't apply to non-browser clients (curl, mobile apps) — they ignore CORS headers

- ▶ Open `http://localhost:3000` — the frontend loads from Express
- ▶ Remove `cors()` — watch the browser block cross-origin requests
- ▶ Add it back — requests work again

What You Can Do Now

- ▶ Build a REST API with Express and TypeScript ✓
- ▶ Use middleware for logging, parsing, and error handling ✓
- ▶ Validate request data with Zod schemas ✓
- ▶ Organize routes with Router ✓
- ▶ Configure CORS for cross-origin requests ✓

The Mental Model

- ▶ A REST API = nouns as URLs, verbs as HTTP methods, JSON in/out
- ▶ Express is the routing layer. Middleware adds cross-cutting behavior.
- ▶ Validate everything at the boundary (Zod)
- ▶ CORS = which origins your API trusts. Lock it down in production.

Next: Prisma + Postgres. We swap the in-memory store for a real database. Same routes, real persistence.